

Training Guide

Cybersecurity Temporal Threat Correlation — build, train, and produce a deployable network

A complete, copy-paste walkthrough of the training half of the pipeline: install the toolchain, build the platform, run the demo, train the model on reference and real datasets, and produce the ready-to-deploy network the Deployment Guide consumes.

Document	Model Training Guide
Product	Jneopallium Cybersecurity Module (Demo 06)
Model	cybersecurity-temporal-threat-correlator 1.0.0
Trainer	scripts/demo-cybersecurity-training/ train_temporal_model.py
Output	Deployable JNeopallium layer/neuron config
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1. What you will achieve
2. Where training ends and deployment begins
3. Prerequisites and system requirements
4. Step 1 — Get the source and build
5. Step 2 — Run the cybersecurity demo
6. Step 3 — Understand the generated demo outputs
7. Step 4 — Train the reference model
8. Step 5 — Train at production scale
9. Step 6 — Train on external datasets (manifest mode)
10. Step 7 — Train on raw LANL + ToN_IoT
11. Step 8 — The generated artifacts: a deployable network
12. Step 9 — Verify the run against the gates
13. Troubleshooting
14. Appendix — training cheat-sheet

1 What you will achieve

By the end of this guide you will have: a built Jneopallium platform; a runnable cybersecurity demo producing advisory JSONL; a freshly trained temporal threat-correlation model; and — the key output — a **complete, deployable JNeopallium network** (layer and neuron configuration files with real runtime classes, embedded trained weights, and a fixed safety gate) that the companion **Deployment Guide** loads directly.

Safety ceiling for the whole pipeline

Everything the trained model does operates in ADVISORY mode. It may recommend investigation or quarantine candidates, but it must not isolate hosts or block traffic. The trainer encodes this directly: the response layer is marked ADVISORY and the hard gate is fixed configuration, never learned.

2 Where training ends and deployment begins

The pipeline has two clean halves, and this guide covers the first one:

Phase	What happens	Document
Training (this guide)	Build, run the demo, train on data, emit the deployable network	Training Guide
Deployment	Load the emitted network, configure event sources, run the advisory worker	Deployment Guide

The hand-off between the two is a single directory of generated artifacts (Step 8). Training **produces** it; deployment **consumes** it. Nothing is hand-authored between them — the trainer writes JNeopallium-ready layer and neuron configuration, so the same files that record your training run are the files the worker boots from.

3 Prerequisites and system requirements

Toolchain

Tool	Version	Why
Java JDK	17 or newer (LTS)	The platform targets the Java 17 baseline API set
Apache Maven	3.9 or newer	Multi-module build of master + worker
Python	3.10+ (3.13 tested)	Runs the model trainer; no third-party packages required
Git	any recent	Clone the repository

Hardware

- **Demo / pilot:** any modern laptop. The reference trainer runs comfortably in a single JVM and the desktop-scale dataset is a few tens of megabytes.
- **Production-scale reference run:** the trainer records a 100 GiB *logical* corpus target without writing 100 GiB to disk, so it still runs on a workstation. Real external-dataset training scales with the data you feed it.

Verify the toolchain

```
java -version      # expect 17 or newer
mvn -version       # expect 3.9 or newer
python --version   # expect 3.10 or newer
git --version
```

4 Step 1 — Get the source and build

Clone the repository and build both modules. The build installs the master and worker artifacts into your local Maven repository.

```
git clone https://github.com/rakovpublic/jneopallium.git
cd jneopallium
mvn clean install
```

A successful build compiles the platform, runs the test suite (including the cybersecurity SecurityModuleTest), and installs com.rakovpublic.jneopallium:worker:1.0. The worker already contains the real security neuron, processor, and signal classes the trained network will reference, so no extra hand-written classes are required to train.

5 Step 2 — Run the cybersecurity demo

The full-run demo suite launches through the real worker entry path. Run the whole suite (Demo 06 is the cybersecurity scenario):

Windows (PowerShell)

```
powershell -ExecutionPolicy Bypass -File scripts/demo-fullrun/run_all_fullrun_demos.ps1
```

Linux / macOS (bash)

```
scripts/demo-fullrun/run_all_fullrun_demos.sh
```

The cybersecurity scenario correlates authentication, process, DNS, network-flow, asset, threat-intelligence, and maintenance context for three entities — an ordered attack chain, a benign maintenance pattern, and a low-and-slow exfiltration pattern — and writes advisory results without taking any blocking action.

6 Step 3 — Understand the generated demo outputs

Default full-run evidence is written under `target/`:

```
target/jneopallium-fullrun-demos/summary.json
target/jneopallium-fullrun-demos/demo-06-cybersecurity-kafka-triage/
```

The Demo 06 output is advisory JSONL. Every line carries the posterior probability, impact, sequence confidence, the threat-intelligence gate, the maintenance gate, and the baseline-freeze state. The expected, deterministic behaviour is:

Stream	Expected verdict
Ordered attack chain	TEMPORAL_THREAT_ADVISORY (baseline frozen during the attack window)
Maintenance-window activity	CONTEXT_SUPPRESSED_OBSERVATION (scored lower, still audited)
Low-and-slow outbound	LOW_AND_SLOW_CORRELATION
Any stream	No active blocking result is ever emitted

7 Step 4 — Train the reference model

The trainer needs no third-party Python packages. The simplest run uses the bundled, deterministic multi-source reference corpus:

```
python scripts/demo-cybersecurity-training/train_temporal_model.py
```

It emits the full artifact set into the worker resources (see Step 8 for the complete list and what each file is). The trainer prints a JSON summary on completion, including the chosen decision threshold, the held-out test F1, and the estimated corpus size. The run fails (non-zero exit) if the test F1 falls below the configured `--min-test-f1` gate (default 0.85), so a broken pipeline cannot silently ship a weak model.

8 Step 5 — Train at production scale

This run records a 100 GiB *logical* corpus target by deterministically replicating the reference campaigns, while keeping the fitted sample bounded so it still completes on a workstation. It exercises the scaling metadata and guardrails.

Windows (PowerShell)

```
python scripts\demo-cybersecurity-training\train_temporal_model.py `
  --reference-multiplier 1000 `
  --target-corpus-bytes 100gb `
  --max-corpus-bytes 100gb `
  --max-train-windows-per-epoch 4096
```

Linux / macOS (bash)

```
python scripts/demo-cybersecurity-training/train_temporal_model.py \
```

```
--reference-multiplier 1000 \
--target-corpus-bytes 100gb \
--max-corpus-bytes 100gb \
--max-train-windows-per-epoch 4096
```

What the flags mean

--reference-multiplier deterministically expands the fitted sample. --target-corpus-bytes is the logical production-scale target recorded in the artifacts. --max-corpus-bytes is a hard guardrail: the run aborts rather than inflate the repository. --max-train-windows-per-epoch caps per-epoch work for very large expanded corpora.

9 Step 6 — Train on external datasets (manifest mode)

For real performance you must train on external canonical data. Manifest mode keeps the same leakage-safe split policy while reading your own JSONL or CSV sources.

```
python scripts\demo-cybersecurity-training\train_temporal_model.py `
--manifest scripts\demo-cybersecurity-training\dataset-manifest-template.json `
--output-dir worker\src\main\resources\model\cybersecurity-temporal `
--max-corpus-bytes 100gb
```

The canonical event contract

Every training row must be canonical JSONL or CSV with these fields:

```
dataset, entity_id, event_tick, source, event_type, technique,
evidence_confidence, threat_intel_confidence, asset_criticality,
maintenance_active, malicious, campaign_id, host_group, attack_type, split
```

Rule	Why it matters
event_tick is event time, not ingestion time	Lets the engine rebuild the true attack sequence
campaign_id groups one incident or matched benign period	Keeps related events together for splitting
split is by time / campaign / host / attack type	Never randomise rows — that leaks duplicates and inflates scores
maintenance_active is a soft context feature	It lowers urgency but must never delete evidence
malicious is ground truth for training only	Runtime inference must never depend on it

10 Step 7 — Train on raw LANL + ToN_IoT

A dedicated template accepts the raw public datasets directly. Download the files and place them under the external folders:

```
scripts/demo-cybersecurity-training/external/lanl/
auth.txt.gz proc.txt.gz dns.txt.gz flows.txt.gz redteam.txt.gz
```

```
scripts/demo-cybersecurity-training/external/toniot/  
... extracted ToN_IoT CSV folders ...
```

- LANL Comprehensive Multi-Source Cyber-Security Events: <https://csr.lanl.gov/data/cyber1/>
- ToN_IoT datasets: <https://research.unsw.edu.au/projects/toniot-datasets>

Then run the LANL + ToN_IoT template (stream the compressed files rather than loading them into memory):

```
python scripts\demo-cybersecurity-training\train_temporal_model.py `  
--manifest scripts\demo-cybersecurity-training\dataset-manifest-lanl-toniot-  
template.json `  
--output-dir worker\src\main\resources\model\cybersecurity-temporal `  
--max-corpus-bytes 100gb `  
--max-train-windows-per-epoch 4096
```

The template also accepts the raw LANL files (lanl-auth, lanl-proc, lanl-dns, lanl-flow, lanl-redteam) and common ToN_IoT CSV files (toniot-network, toniot-windows, toniot-linux). Tune maxRows, startTick, endTick, and sampleModulo in the manifest before training on large extracts. Keep dataset licences and download instructions outside generated artifacts.

11 Step 8 — The generated artifacts: a deployable network

This is the pipeline's hand-off. Every training run writes a directory of JNeopallium-style configuration that is ready to deploy as-is — not just an abstract model, but the actual layer-and-neuron network the worker boots from. Ten files are produced:

Artifact	What it is
trained-temporal-threat-model.json	The compact trained model: features, scaler, weights, bias, threshold, metrics
model-descriptor.json	The whole-network descriptor: 5 layers, 8 neurons, frequency map, source classes
layer-0.json	Multi-source security event input boundary (canonical inputs per source family)
layer-1-fast-evidence.json	Fast evidence receptors — 4 neurons (flow, signature, process, baseline)
layer-2-temporal-correlation.json	The trained temporal correlator — embeds weights, gates, dendrites
layer-3-response-planning.json	Advisory response planning + the fixed hard safety gate
result-layer.json	The security-advisory result output layer
trained-model-update.json	Latest trained snapshot: status, checksum, weights, training summary
quantitative-summary.json	Scale, metrics, and top positive / negative feature weights
source-mapping.json	Which typed signals each data source maps onto

The network the trainer builds

`model-descriptor.json` records a five-layer network of eight real neurons, with 34 trainable weights and one bias — all referencing concrete runtime classes from the worker's security package:

Layer	Role	Real neuron classes
0 — Input	Multi-source event boundary	(no neuron objects; canonical inputs only)
1 — Fast evidence	Cheap, instant scoring	NetworkFlowNeuron, SignaturePatternNeuron, ProcessBehaviourNeuron, EntityBehaviourBaselineNeuron
2 — Correlation	Trained temporal correlation	TemporalThreatCorrelationNeuron
3 — Planning + gate	Advisory bands + safety gate	ResponsePlanningNeuron, ResponseGateNeuron
4 — Result	Advisory output	ResponsePlanningNeuron (result)

The correlation layer (`layer-2-temporal-correlation.json`) embeds the trained logistic weights as named **dendrite weights**, the standardisation scaler, the decision threshold (0.2), the five temporal sequence gates, and a `baselineAdaptation` block that freezes learning when the posterior reaches **0.3** or signature confidence reaches **0.8**, and only adapts from trusted benign periods. The descriptor also carries a `signalFrequencyMap` giving each signal its loop cadence — fast receptors every cycle, hypotheses and quarantine requests every second cycle, incident reports every fifth — the typed-timescale design made concrete.

Why this matters

Because training emits real, inspectable, deployable configuration, there is no translation step between "the model we trained" and "the network we run." The Deployment Guide simply points the worker at this directory. Every weight, gate, and threshold you see in the report is exactly what executes in production.

12 Step 9 — Verify the run against the gates

Before treating any run as deployable, confirm the production gates pass:

1. All expected source families are present in `source-mapping.json`.
2. Test F1 is at least the configured `--min-test-f1`.
3. False-positive rate is within the per-host and per-tenant budget.
4. Calibration bins show monotonic risk ordering (higher score → higher real positive rate).
5. Mean time to detection is below your incident-response objective.
6. The five layer files and `trained-model-update.json` were written and reference the expected runtime classes.
7. Generated artifacts include checksum, source counts, split policy, metrics, and threshold.

Reproducibility check

The reference run is deterministic. Re-running the same command must produce the same training checksum (trained-model-update.json) and the same metrics. A drift in either is a signal that an input or the toolchain changed.

13 Troubleshooting

Symptom	Likely cause and fix
java -version shows < 17	Install a JDK 17+ and set JAVA_HOME to it
Build fails on JDK 17 with reflection warnings	Ensure Maven plugin floors from the README; use <release>17</release>
Trainer exits non-zero with "test F1 below required"	The split or data is too weak; check the manifest split policy and --min-test-f1
"estimated canonical corpus exceeds --max-corpus-bytes"	Lower --reference-multiplier/--target-corpus-bytes or raise the guardrail deliberately
A source is missing in source-mapping.json	Check manifest globs and requireAllSources; confirm files were downloaded
Layer files not written	Confirm --output-dir is writable; the trainer writes all ten artifacts there
Cyrillic / encoding issues in CSV	Save sources as UTF-8; the reader expects UTF-8 JSONL/CSV

14 Appendix — training cheat-sheet

Build

```
git clone https://github.com/rakovpublic/jneopallium.git && cd jneopallium && mvn clean install
```

Run the demo suite

```
# Windows
powershell -ExecutionPolicy Bypass -File scripts/demo-fullrun/run_all_fullrun_demos.ps1
# Linux / macOS
scripts/demo-fullrun/run_all_fullrun_demos.sh
```

Train (reference / scaled / external / raw)

```
# reference
python scripts/demo-cybersecurity-training/train_temporal_model.py

# production-scale reference
python scripts/demo-cybersecurity-training/train_temporal_model.py \
  --reference-multiplier 1000 --target-corpus-bytes 100gb \
  --max-corpus-bytes 100gb --max-train-windows-per-epoch 4096

# external manifest
```

```
python scripts/demo-cybersecurity-training/train_temporal_model.py \  
  --manifest scripts/demo-cybersecurity-training/dataset-manifest-template.json  
  
# raw LANL + ToN_IoT  
python scripts/demo-cybersecurity-training/train_temporal_model.py \  
  --manifest scripts/demo-cybersecurity-training/dataset-manifest-lanl-toniot-  
template.json \  
  --max-corpus-bytes 100gb --max-train-windows-per-epoch 4096
```

Jneopallium Cybersecurity Module · Demo 06 · Training Guide. The trainer emits a deployable JNeopallium network; see the Deployment Guide to run it. Safety mode: ADVISORY. License: BSD 3-Clause.